

# Chaîne Unique

Une seule machine, des délais fermes, et des pénalités que personne ne sait expliquer.



## Contexte et objectif

---

Un atelier de reprographie industrielle imprime, façonne et relie des documents à la commande. Il dispose d'une unique chaîne de production :

- Les tâches s'exécutent en série (aucun parallélisme).
- Une tâche lancée ne peut pas être interrompue (pas de préemption).



Chaque commande arrive avec une durée d'exécution, une date limite de livraison ferme et une pénalité financière appliquée en cas de retard. La seule décision possible consiste à choisir l'ordre de passage des commandes.

Actuellement, l'atelier applique l'ordre des échéances (les commandes les plus urgentes d'abord). Chaque fin de mois, des pénalités tombent sans que personne ne sache si ce montant était évitable.

Votre objectif n'est pas d'atteindre une perfection absolue — ce qui est souvent impossible en temps raisonnable —, mais de quantifier précisément l'écart entre différentes stratégies et l'optimum théorique, puis de recommander la règle que l'atelier appliquera concrètement en production.

## Cadre de travail et contraintes techniques

---

**Organisation :** Projet individuel avec un dépôt GitHub dédié.

**Langages autorisés :** C++ (17 ou plus), Java (17 ou plus) ou TypeScript (Node.js). Le choix est définitif.

**Mesures de performance :** Les comparaisons reposent exclusivement sur des compteurs algorithmiques (nombre de commandes sauvées, somme des pénalités), jamais sur des durées d'exécution en secondes.



| Outils strictement Interdits                                                                                                                                                                                                                         | Outils Autorisés                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Types d'outils :</b>                                                                                                                                                                                                                              | <b>Structures de données &amp; Fonctions :</b>                                                       |
| Solveurs d'optimisation                                                                                                                                                                                                                              | Fonctions de tri standards                                                                           |
| Bibliothèques de programmation linéaire / en nombres entiers                                                                                                                                                                                         | Files de priorité ( <a href="#">PriorityQueue</a> , <a href="#">std::priority_queue</a> )            |
| Moteurs d'ordonnancement externes                                                                                                                                                                                                                    | Tas binaires (standards ou faits maison)                                                             |
| Appels système à des solveurs externes                                                                                                                                                                                                               | Tables de hachage, conteneurs et listes                                                              |
| <b>Bibliothèques spécifiquement exclues :</b><br>org.apache.commons.math3.optim.linear<br><br>• JSprit, jMetal, OptaPlanner<br><br>• SCIP, HiGHS, CBC, Clp, LEMON<br><br>• highs-js, javascript-lp-solver<br><br>• jsLPSolver, linear-program-solver | <b>Remarque :</b> Ce sont des structures de données fondamentales et non des solveurs d'optimisation |

## Spécifications métier et format des fichiers

### Modèle d'une commande

Chaque commande est un triplet d'entiers strictement positifs :

- **p** : durée de fabrication.
- **d** : échéance (date limite de livraison).
- **w** : pénalité contractuelle due si l'échéance est dépassée.



## Règles de fonctionnement de l'atelier

- La chaîne démarre à l'instant 0 et tourne sans temps mort entre deux commandes.
- Une commande est livrée à l'heure si son instant de fin est inférieur ou égal à **d**.
- En cas de dépassement, même minime, la pénalité **w** est due intégralement : un retard d'une minute entraîne la même sanction qu'un retard d'une semaine.

## Règle de départage imposée

Quand une règle doit retirer une commande parmi plusieurs candidates ex æquo :

1. Retirer la commande ayant la plus grande durée **p**.
2. En cas d'égalité sur la durée, retirer celle dont la ligne apparaît le plus bas dans le fichier source.

Cette règle garantit une reproductibilité parfaite des calculs et la validité des oracles de test.

## Format du carnet de commandes

Le fichier texte contient une commande par ligne (identifiant sans espace, séparé par des espaces) :



SIGLE duree echeance penalite

## Étapes du projet

---

### Étape 1 : Le générateur de carnets

Vous devez concevoir votre propre générateur afin de maîtriser les propriétés des données testées :

- **Générateur pseudo-aléatoire maison :** Un générateur 32 bits reproductible piloté par une graine (graine entière, par exemple xorshift32 ou congruentiel modulo  $2^{32}$ ).
  - En TypeScript : manipuler les entiers avec Math.imul et clore chaque opération par `>>> 0` pour éviter les pertes de précision.
  - En Java : employer l'opérateur de décalage non signé `>>>` au lieu de `>>`.
- **Plantation d'une solution témoin :**
  - Générer d'abord un sous-ensemble de commandes garanti livrable à l'heure par construction (durées choisies et cumulées, échéance au moins égale au cumul d'achèvement).
  - Enregistrer la taille exacte de cet ensemble dans un fichier témoin séparé. Vos algorithmes d'ordonnancement ne doivent jamais lire ce fichier : il sert d'étalon de vérification externe.
  - Ajouter des commandes parasites (« bruit ») aux échéances courtes pour saturer la planification.



- Mélanger l'ensemble des commandes. La reproductibilité est assurée par la lecture du fichier généré, pas par la relance de la graine.
- **Visualisation ASCII (la frise)** : Mettre au point une sortie terminal compacte représentant l'axe du temps (un caractère par unité de temps) avec positionnement des échéances. À réserver aux carnets courts ( $\leq 60$  unités cumulées) ; basculer sur un affichage tabulaire au-delà.

## Étape 2 : Le test de faisabilité (l'arbitre)

Avant de chercher à optimiser, écrivez l'outil qui vérifie la validité d'une séquence :

- **Théorème fondamental** : Un ensemble de commandes est intégralement livrable à l'heure si et seulement si l'ordonnancement par échéances croissantes (**d** croissant) ne génère aucun retard.
- **Justification écrite** : Expliquer la validité de ce principe via un argument d'échange (montrer qu'inverser deux commandes consécutives qui ne respectent pas l'ordre des échéances ne détériore pas la faisabilité).
- **Implémentation** : Cinq lignes suffisent : trier par échéances, cumuler les durées et vérifier que le cumul reste sous l'échéance à chaque étape.
- **Propriété clé à formuler** : La faisabilité d'un ensemble dépend uniquement des durées **p** et des échéances **d**, jamais des pénalités **w**.

## Étape 3 : Implémentation des quatre règles

Les quatre stratégies reposent sur une structure identique d'une dizaine de lignes : trier par échéances croissantes, parcourir les commandes en cumulant le temps, et dès qu'un retard apparaît, éjecter définitivement une commande de la sélection en cours.



- **Règle R1** : Ne retire aucune commande. Elle applique l'ordre des échéances sans ajustement (pratique actuelle de l'atelier, sert de témoin).
- **Règle R2** : Éjecte la commande ayant la plus longue durée parmi les commandes retenues jusqu'ici. Vise à maximiser le nombre total de commandes livrées à l'heure.
- **Règle R3** : Éjecte la commande ayant la plus faible pénalité  $w$ . Vise à maximiser la valeur financière livrée à l'heure.
- **Règle R3'** : Éjecte la commande ayant le plus faible ratio pénalité sur durée ( $w/p$ ). Variante heuristique de R3.

#### Attendus théoriques :

- Rédiger la démonstration formelle (environ une demi-page) prouvant l'optimalité de R2 pour le nombre de commandes sauvées, en vous appuyant sur un argument d'échange.
- Analyser la différence de statut mathématique : R2 fournit une solution garantie optimale, tandis que R3 et R3' ne disposent d'aucune garantie (il n'existe aucun algorithme polynomial exact connu pour ce critère).

### Étape 4 : Borne théorique, vérificateur exact et campagne de tests

- **Calcul de la borne** : Comme la faisabilité ne dépend pas des pénalités, aucun planning ne peut sauver plus de  $k$  commandes (valeur optimale exacte calculée par R2). On en déduit :
  - Pénalités sauvées \ somme des  $k$  plus grandes pénalités du carnet.
  - Pénalités payées \ pénalités totales - cette même somme.*Rédiger la preuve en trois lignes et son calcul en cinq lignes.*
- **Vérificateur exhaustif** : Sur des carnets restreints (16 à 18 commandes), implémenter une recherche combinatoire exhaustive (arbre binaire prendre / laisser), évaluer chaque sous-ensemble via le juge de l'étape



2 et retenir le meilleur résultat absolu. Cela permet de mesurer l'erreur réelle de vos heuristiques et l'écart de la borne.

- **Validation par oracles :** Votre programme doit valider automatiquement les invariants suivants :
  - Si toutes les pénalités valent 1, R2 et R3 retournent exactement le même nombre de commandes à l'heure.
  - Le nombre de commandes à l'heure de R1 est toujours inférieur ou égal à celui de R2.
  - Le nombre de commandes à l'heure de R3 est toujours inférieur ou égal à celui de R2.
  - Le nombre de commandes à l'heure de R2 est toujours supérieur ou égal au nombre de commandes plantées lors de la génération.
  - Pour chaque règle : pénalités payées + pénalités sauvées = somme totale des pénalités.
- **Campagne d'expérimentation (export CSV) :**

Générer des séries de tests (jusqu'à 2 000 commandes par carnet) sur trois familles de données :

  - Pénalités indépendantes de la durée.
  - Pénalités corrélées positivement aux durées (grosses commandes à fort enjeu).
  - Pénalités anti-corrélées aux durées (tâches courtes et très coûteuses en cas de retard).

Chaque configuration doit présenter la moyenne sur au moins 50 exécutions, accompagnée de la valeur du pire cas observé.

## Étape 5 : Carnets adverses et arbitrage final

- **Mise en échec de R3 :** Construire une famille de cas paramétrée par un entier **n** où les résultats de R3 se détériorent sans borne :
  - Une commande longue **A** : durée **n**, échéance **n**, pénalité 10.



- **n** commandes courtes : durée 1, échéance **n** + 1, pénalité 9.  
R3 conserve **A** et une seule commande courte (score : 19), alors que traiter l'ensemble des commandes courtes rapporte **9n**.  
Vérifier l'écart avec votre juge sur au moins trois valeurs distinctes de **n**.
- **Comportement de R3'** : Montrer comment la prise en compte du ratio permet à R3' d'éjecter immédiatement la commande **A** et d'obtenir l'optimum sur cette même famille.
- Recherche du pire cas pour R3' : Concevoir ou rechercher le carnet qui produit la dégradation la plus marquée sur R3'. Documenter l'instance obtenue, le ratio calculé et justifier la limite atteinte.
- Note de décision (une page) : Rédiger votre recommandation finale argumentée pour le chef d'atelier : quelle règle mettre en place lundi matin, en pesant les performances moyennes observées face aux risques de défaillance sur les cas limites ?

## Livrables attendus

---

- Le générateur pseudo-aléatoire maison avec mécanisme de plantation, ajout de bruit, mélange et fichier témoin indépendant.
- Le test de faisabilité, accompagné de l'énoncé du théorème et de sa démonstration écrite.
- Les quatre règles R1, R2, R3 et R3' basées sur l'architecture commune et respectant le départage imposé.
- La démonstration rédigée prouvant l'optimalité de R2 pour le volume de commandes à l'heure.
- La formalisation mathématique et le calcul de la borne des **k** pénalités.
- Le vérificateur exhaustif pour les instances de 16 à 18 commandes.
- La suite de tests automatisés validant l'ensemble des oracles.



- Les résultats de la campagne de mesures exportés au format CSV (moyennes et pires cas).
- L'étude de la famille adverse paramétrée contre R3 testée sur trois valeurs de  $n$ , et l'évaluation de R3' sur ce jeu.
- L'instance adverse construite contre R3' avec analyse de l'écart mesuré.
- La note de décision d'une page destinée à l'exploitation.
- Le fichier README documentant le choix du langage, les procédures de compilation et d'exécution, la structure des fichiers et les choix d'implémentation.

## Structure du dépôt et conventions de rendu

---

### Organisation obligatoire des répertoires :

```
ChaineUnique/  
  src/           (générateur, juge, règles, vérificateur exhaustif)  
  carnets/       (carnets générés, fichiers témoins, carnets adverses)  
  mesures/       (le CSV de la campagne)  
  docs/          (preuves écrites, note de décision)  
  README.md
```

## Compétences évaluées

---

- **BC01-C3** : Développer des composants métier.
- **BC03-C1** : Préparer et exécuter les plans de tests.
- **BC02-C2** : Définir l'architecture logicielle.



## Pistes d'approfondissement (optionnelles)

---

*Ces extensions s'adressent aux étudiants ayant complété l'intégralité des livrables et validé tous les oracles. Elles ne se substituent pas aux attendus de base.*

- **Résolution exacte par programmation dynamique** : Développer un algorithme exact pseudo-polynomial basé sur l'horizon temporel pour résoudre des carnets plus larges, puis formaliser pourquoi cette approche ne contredit pas le caractère NP-difficile de l'optimisation des pénalités.
- **Extension à deux machines** : Évaluer le comportement de R2 sur un atelier à deux chaînes parallèles (attribution à la première chaîne disponible). Mettre en évidence la perte d'optimalité par un contre-exemple et quantifier l'écart avec l'optimum.
- **Traitement en flux continu (en ligne)** : Concevoir une règle prenant ses décisions au fil de l'eau, sans connaître les commandes futures, et comparer son efficacité par rapport aux traitements hors ligne.

## Base de connaissances

---

### Ordonnancement sur machine unique

- [Règle de Jackson, ordre par échéances croissantes](#)
- [Algorithme de Moore et Hodgson, minimiser le nombre de commandes en retard](#)
- [Notation à trois champs des problèmes d'ordonnancement](#)

### Gloutons et preuves d'optimalité

- [Argument d'échange, la technique de preuve utilisée deux fois cette semaine](#)



- [Cours ouvert de Jeff Erickson, chapitre sur les algorithmes gloutons](#)

## **Complexité**

- [Ce que veut dire « NP-difficile », et ce que cela n'interdit pas](#)
- [Algorithme pseudo-polynomial](#)

## **Générateurs pseudo-aléatoires**

- [Marsaglia, Xorshift RNGs](#)
- [Math.imul, et pourquoi il existe](#)